

Lab Assignment # 05

Course Title : **AI Assistant Coding**
Name of Student : **BG.Sreevani**
Enrollment No. : **2303A54066**
Batch No. : **48**

Lab 5: Ethical Foundations – Responsible AI Coding Practices

Task Description #1 (Privacy in API Usage)

Task: Use an AI tool to generate a Python program that connects to a weather API.

Prompt: "Generate code to fetch weather data securely without exposing API keys in the code."

Expected Output:

- Original AI code (check if keys are hardcoded).
- Secure version using environment variables.

Output Screenshot:

The screenshot shows a Python IDE with a file named `weather.py`. The code is a Python script that fetches weather data from the OpenWeatherMap API. It uses `os.getenv` to retrieve the API key from environment variables, ensuring it is not hardcoded. The script prompts the user for a city name and prints the weather details if found, or an error message if not.

```
class student.py | conditional statements.py | sum.py | design.py | armstrong.py | weather.py 2 x | arr | > ...
weather.py > ...
1 #Task Description #1 (Privacy in API Usage
2 #generate a Python program that connects to aweather APIo fetch weather data securely without exposing API ke
3 #Expected Output: • Original AI code (check if keys are hardcoded).Secure version using environment variables
4
5 import os
6 import requests
7 from dotenv import load_dotenv
8
9 # Load variables from .env file
10 load_dotenv()
11
12 api_key = os.getenv('WEATHER_API_KEY')
13 city = input("Enter city name: ")
14 base_url = "http://api.openweathermap.org/data/2.5/weather?"
15 complete_url = f"{base_url}q={city}&appid={api_key}&units=metric"
16
17 response = requests.get(complete_url)
18 data = response.json()
19
20 if data.get("cod") == 200:
21     main = data["main"]
22     weather_desc = data["weather"][0]["description"]
23     print(f"City: {city}")
24     print(f"Temperature: {main['temp']}°C")
25     print(f"Weather: {weather_desc}")
26 else:
27     print("Error:", data.get("message", "City Not Found"))
28
29
30
31
32
```

The terminal at the bottom shows a `ModuleNotFoundError: No module named 'requests'` error, indicating that the `requests` library is not installed in the current environment.

Task Description #2 (Privacy & Security in File Handling)

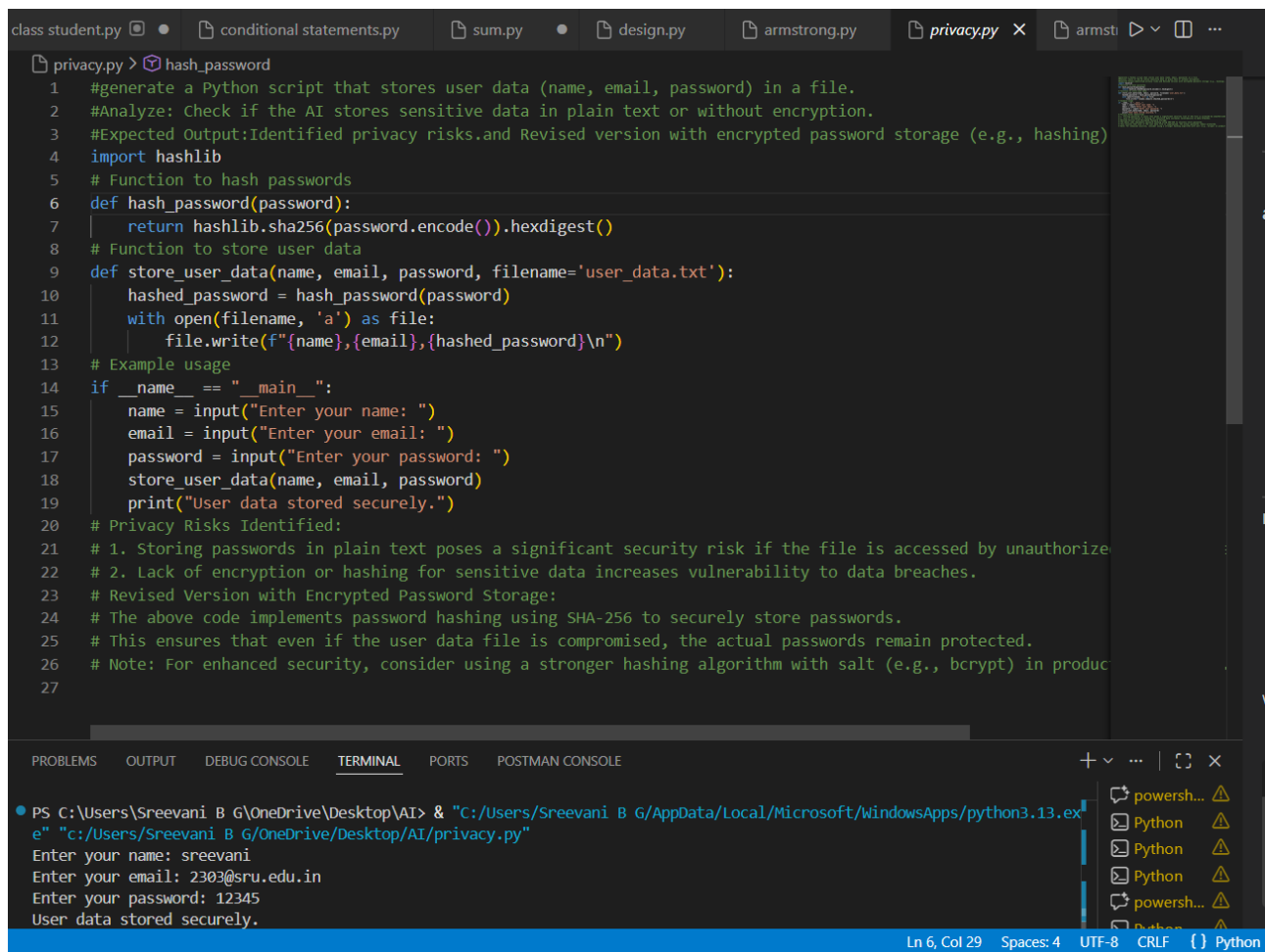
Task: Use an AI tool to generate a Python script that stores user data (name, email, password) in a file.

Analyze: Check if the AI stores sensitive data in plain text or without encryption.

Expected Output:

- Identified privacy risks.
- Revised version with encrypted password storage (e.g., hashing).

Output Screenshot:



```
class student.py • conditional statements.py sum.py • design.py armstrong.py privacy.py × armsti ▶ ▢ ...
privacy.py > hash_password
1 #generate a Python script that stores user data (name, email, password) in a file.
2 #Analyze: Check if the AI stores sensitive data in plain text or without encryption.
3 #Expected Output:Identified privacy risks.and Revised version with encrypted password storage (e.g., hashing)
4 import hashlib
5 # Function to hash passwords
6 def hash_password(password):
7     return hashlib.sha256(password.encode()).hexdigest()
8 # Function to store user data
9 def store_user_data(name, email, password, filename='user_data.txt'):
10     hashed_password = hash_password(password)
11     with open(filename, 'a') as file:
12         file.write(f"{name},{email},{hashed_password}\n")
13 # Example usage
14 if __name__ == "__main__":
15     name = input("Enter your name: ")
16     email = input("Enter your email: ")
17     password = input("Enter your password: ")
18     store_user_data(name, email, password)
19     print("User data stored securely.")
20 # Privacy Risks Identified:
21 # 1. Storing passwords in plain text poses a significant security risk if the file is accessed by unauthorized users.
22 # 2. Lack of encryption or hashing for sensitive data increases vulnerability to data breaches.
23 # Revised Version with Encrypted Password Storage:
24 # The above code implements password hashing using SHA-256 to securely store passwords.
25 # This ensures that even if the user data file is compromised, the actual passwords remain protected.
26 # Note: For enhanced security, consider using a stronger hashing algorithm with salt (e.g., bcrypt) in production.
27

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
• PS C:\Users\Sreevani B G\OneDrive\Desktop\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/Desktop/AI/privacy.py"
Enter your name: sreevani
Enter your email: 2303@srj.edu.in
Enter your password: 12345
User data stored securely.
Ln 6, Col 29 Spaces: 4 UTF-8 CRLF {} Python
```

Task Description #3 (Transparency in Algorithm Design)

Task: Use AI to generate an Armstrong number checking function with comments and explanations.

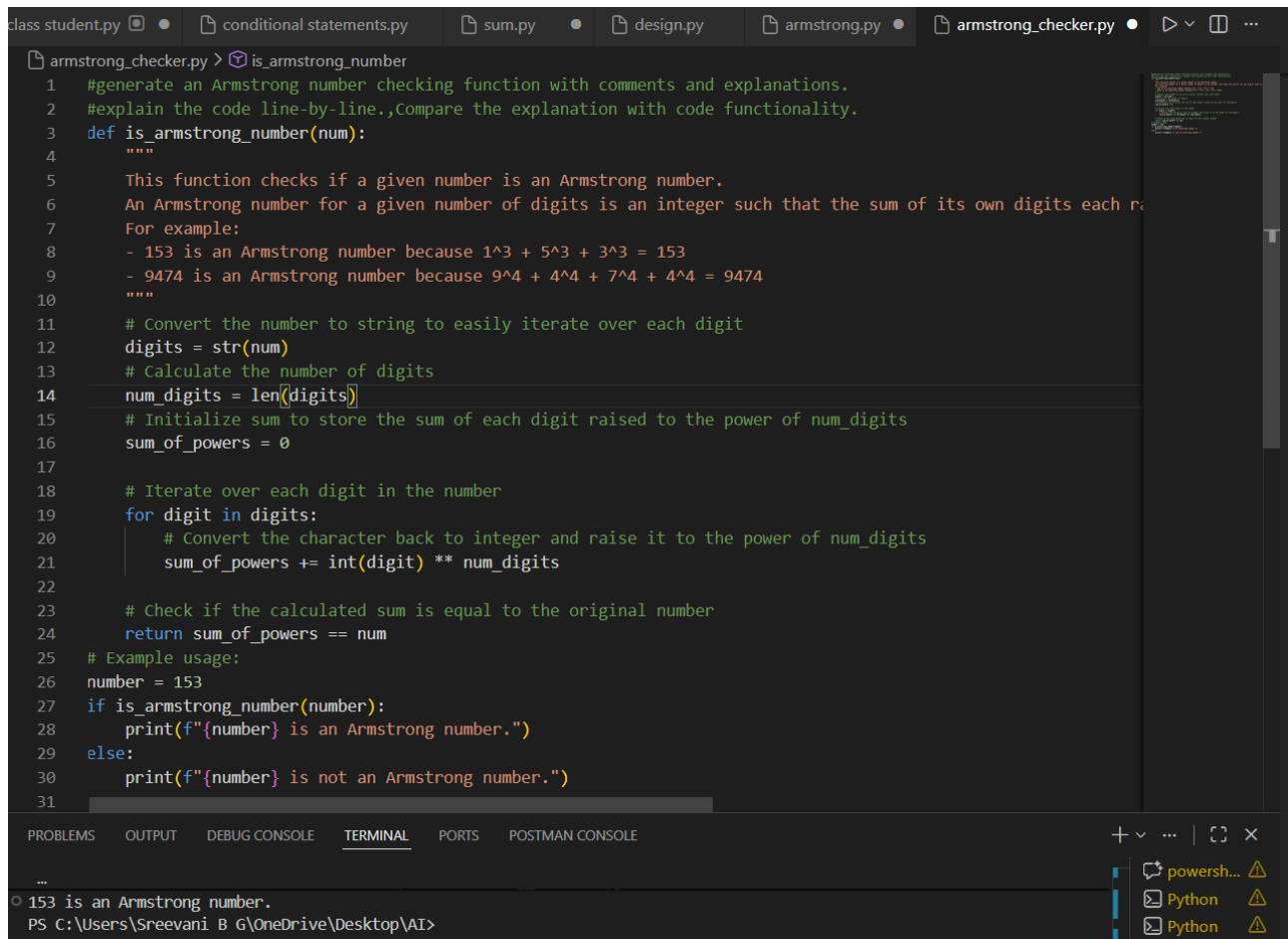
Instructions:

1. Ask AI to explain the code line-by-line.
2. Compare the explanation with code functionality.

Expected Output:

- Transparent, commented code.
- Correct, easy-to-understand explanation.

Output Screenshot:



```
class student.py • conditional statements.py • sum.py • design.py • armstrong.py • armstrong_checker.py • ▶ ▢ ...
armstrong_checker.py > is_armstrong_number
1 #generate an Armstrong number checking function with comments and explanations.
2 #explain the code line-by-line.,Compare the explanation with code functionality.
3 def is_armstrong_number(num):
4     """
5     This function checks if a given number is an Armstrong number.
6     An Armstrong number for a given number of digits is an integer such that the sum of its own digits each raised to the power of the number of digits is equal to the number itself.
7     For example:
8     - 153 is an Armstrong number because 1^3 + 5^3 + 3^3 = 153
9     - 9474 is an Armstrong number because 9^4 + 4^4 + 7^4 + 4^4 = 9474
10    """
11    # Convert the number to string to easily iterate over each digit
12    digits = str(num)
13    # Calculate the number of digits
14    num_digits = len(digits)
15    # Initialize sum to store the sum of each digit raised to the power of num_digits
16    sum_of_powers = 0
17
18    # Iterate over each digit in the number
19    for digit in digits:
20        # Convert the character back to integer and raise it to the power of num_digits
21        sum_of_powers += int(digit) ** num_digits
22
23    # Check if the calculated sum is equal to the original number
24    return sum_of_powers == num
25
26    # Example usage:
27    number = 153
28    if is_armstrong_number(number):
29        print(f"{number} is an Armstrong number.")
30    else:
31        print(f"{number} is not an Armstrong number.")
32
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
...
153 is an Armstrong number.
PS C:\Users\Sreevani B G\OneDrive\Desktop\AI>
```

Task Description #4 (Transparency in Algorithm Comparison)

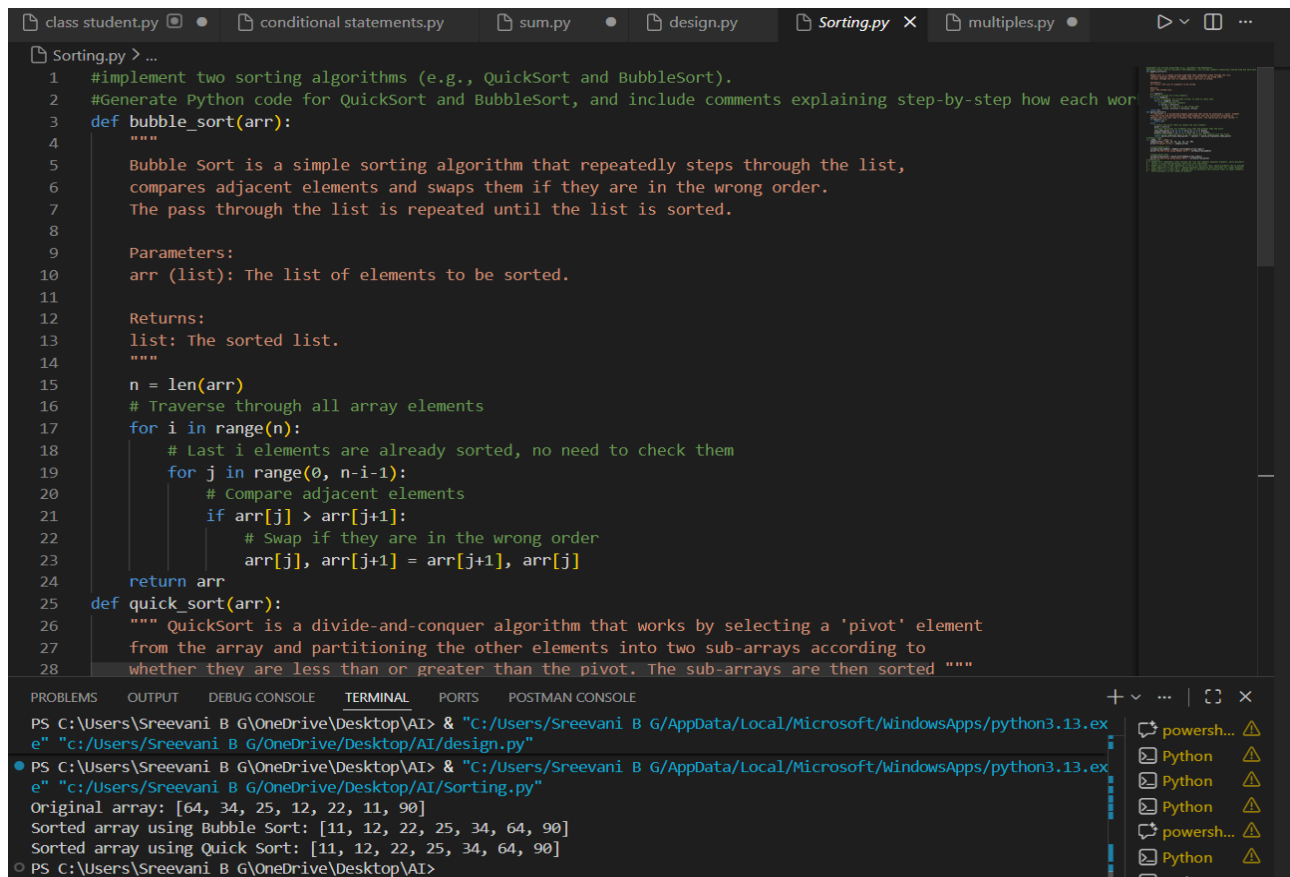
Task: Use AI to implement two sorting algorithms (e.g., QuickSort and BubbleSort).

Prompt: "Generate Python code for QuickSort and BubbleSort, and include comments explaining step-by-step how each works and where they differ."

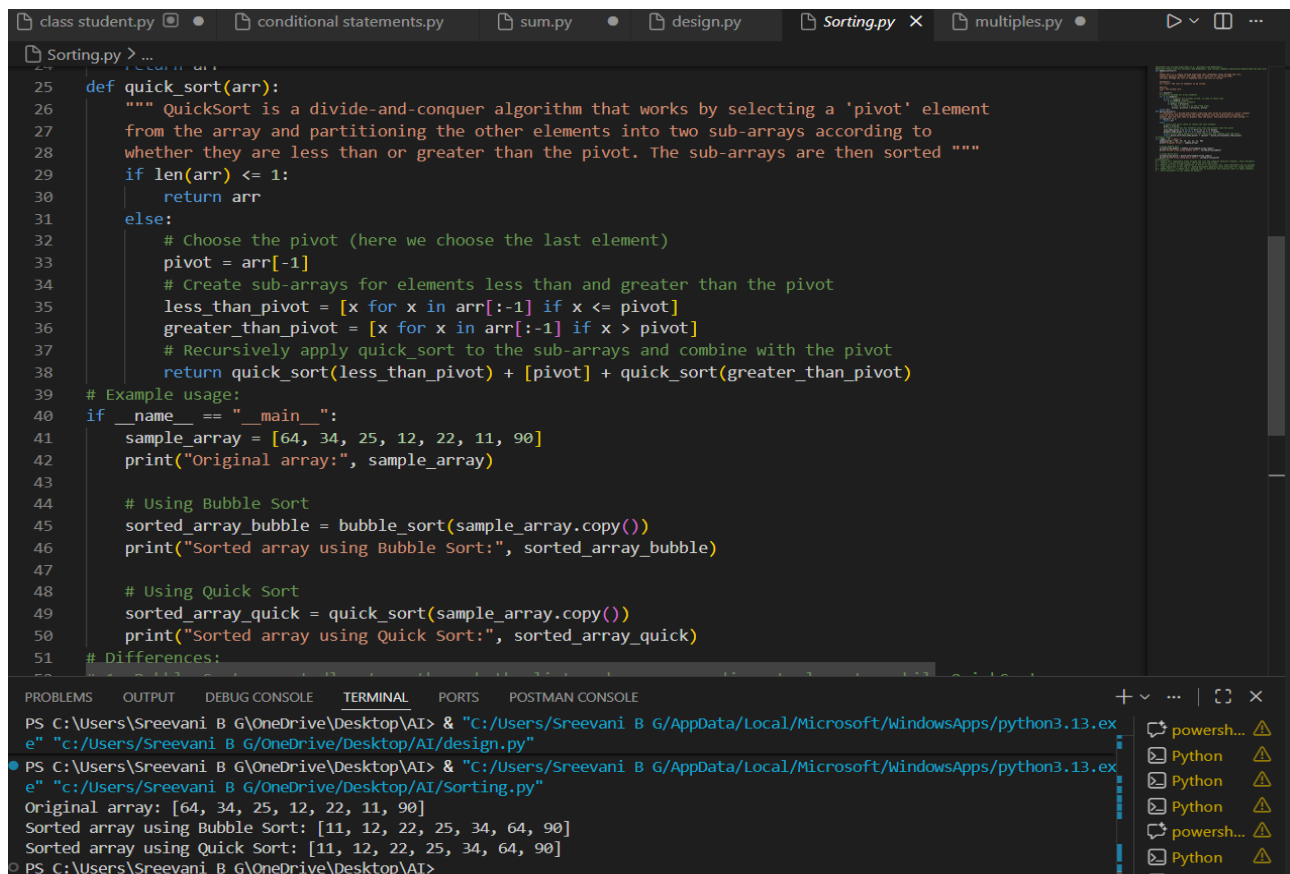
Expected Output:

- Code for both algorithms.
- Transparent, comparative explanation of their logic and efficiency.

Output Screenshot:



```
class student.py • conditional statements.py sum.py • design.py Sorting.py x multiples.py
Sorting.py > ...
1 #implement two sorting algorithms (e.g., QuickSort and BubbleSort).
2 #Generate Python code for QuickSort and BubbleSort, and include comments explaining step-by-step how each wor
3 def bubble_sort(arr):
4     """
5     Bubble Sort is a simple sorting algorithm that repeatedly steps through the list,
6     compares adjacent elements and swaps them if they are in the wrong order.
7     The pass through the list is repeated until the list is sorted.
8
9     Parameters:
10    arr (list): The list of elements to be sorted.
11
12    Returns:
13    list: The sorted list.
14    """
15    n = len(arr)
16    # Traverse through all array elements
17    for i in range(n):
18        # Last i elements are already sorted, no need to check them
19        for j in range(0, n-i-1):
20            # Compare adjacent elements
21            if arr[j] > arr[j+1]:
22                # Swap if they are in the wrong order
23                arr[j], arr[j+1] = arr[j+1], arr[j]
24    return arr
25 def quick_sort(arr):
26     """ QuickSort is a divide-and-conquer algorithm that works by selecting a 'pivot' element
27     from the array and partitioning the other elements into two sub-arrays according to
28     whether they are less than or greater than the pivot. The sub-arrays are then sorted """
29
30 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\Sreevani B G\OneDrive\Desktop\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "C:/Users/Sreevani B G/OneDrive/Desktop/AI/design.py"
PS C:\Users\Sreevani B G\OneDrive\Desktop\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "C:/Users/Sreevani B G/OneDrive/Desktop/AI/Sorting.py"
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array using Bubble Sort: [11, 12, 22, 25, 34, 64, 90]
Sorted array using Quick Sort: [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\Sreevani B G\OneDrive\Desktop\AI>
```



```
class student.py • conditional statements.py sum.py • design.py Sorting.py x multiples.py
Sorting.py > ...
25 def quick_sort(arr):
26     """ QuickSort is a divide-and-conquer algorithm that works by selecting a 'pivot' element
27     from the array and partitioning the other elements into two sub-arrays according to
28     whether they are less than or greater than the pivot. The sub-arrays are then sorted """
29     if len(arr) <= 1:
30         return arr
31     else:
32         # Choose the pivot (here we choose the last element)
33         pivot = arr[-1]
34         # Create sub-arrays for elements less than and greater than the pivot
35         less_than_pivot = [x for x in arr[:-1] if x <= pivot]
36         greater_than_pivot = [x for x in arr[:-1] if x > pivot]
37         # Recursively apply quick_sort to the sub-arrays and combine with the pivot
38         return quick_sort(less_than_pivot) + [pivot] + quick_sort(greater_than_pivot)
39 # Example usage:
40 if __name__ == "__main__":
41     sample_array = [64, 34, 25, 12, 22, 11, 90]
42     print("Original array:", sample_array)
43
44     # Using Bubble Sort
45     sorted_array_bubble = bubble_sort(sample_array.copy())
46     print("Sorted array using Bubble Sort:", sorted_array_bubble)
47
48     # Using Quick Sort
49     sorted_array_quick = quick_sort(sample_array.copy())
50     print("Sorted array using Quick Sort:", sorted_array_quick)
51 # Differences:
52
53 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
PS C:\Users\Sreevani B G\OneDrive\Desktop\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "C:/Users/Sreevani B G/OneDrive/Desktop/AI/design.py"
PS C:\Users\Sreevani B G\OneDrive\Desktop\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "C:/Users/Sreevani B G/OneDrive/Desktop/AI/Sorting.py"
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array using Bubble Sort: [11, 12, 22, 25, 34, 64, 90]
Sorted array using Quick Sort: [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\Sreevani B G\OneDrive\Desktop\AI>
```

Key Differences

- QuickSort is much faster in practice because it reduces the problem size at each step by partitioning the array.
- BubbleSort is conceptually simple but inefficient, as it repeatedly scans the array and swaps elements, leading to quadratic time complexity.
- QuickSort is widely used in real-world applications, while BubbleSort is mostly used for educational purposes to demonstrate

◇ Efficiency Comparison

	$O(n \log n)$	$O(n)$
	$O(n \log n)$	$O(n^2)$
	$O(n^2)$	$O(n^2)$
	$O(\log n)$	$O(1)$

◇ Transparent Takeaways

- QuickSort is efficient and scalable:
It reduces the sorting problem size at each step, making it suitable for large datasets. Its recursive nature leverages divide-and-conquer, which is why it's widely used in practice.
- BubbleSort is simple but inefficient:
It's easy to understand and implement, but its repeated passes make it extremely slow for large inputs. It's mainly used for teaching sorting concepts, not real-world applications.
- Key Difference in Logic:
QuickSort divides the array into smaller problems, while BubbleSort repeatedly pushes larger elements to the end.
This difference in strategy explains why QuickSort achieves logarithmic efficiency while BubbleSort suffers quadratic inefficiency

Task Description #5 (Transparency in AI Recommendations)

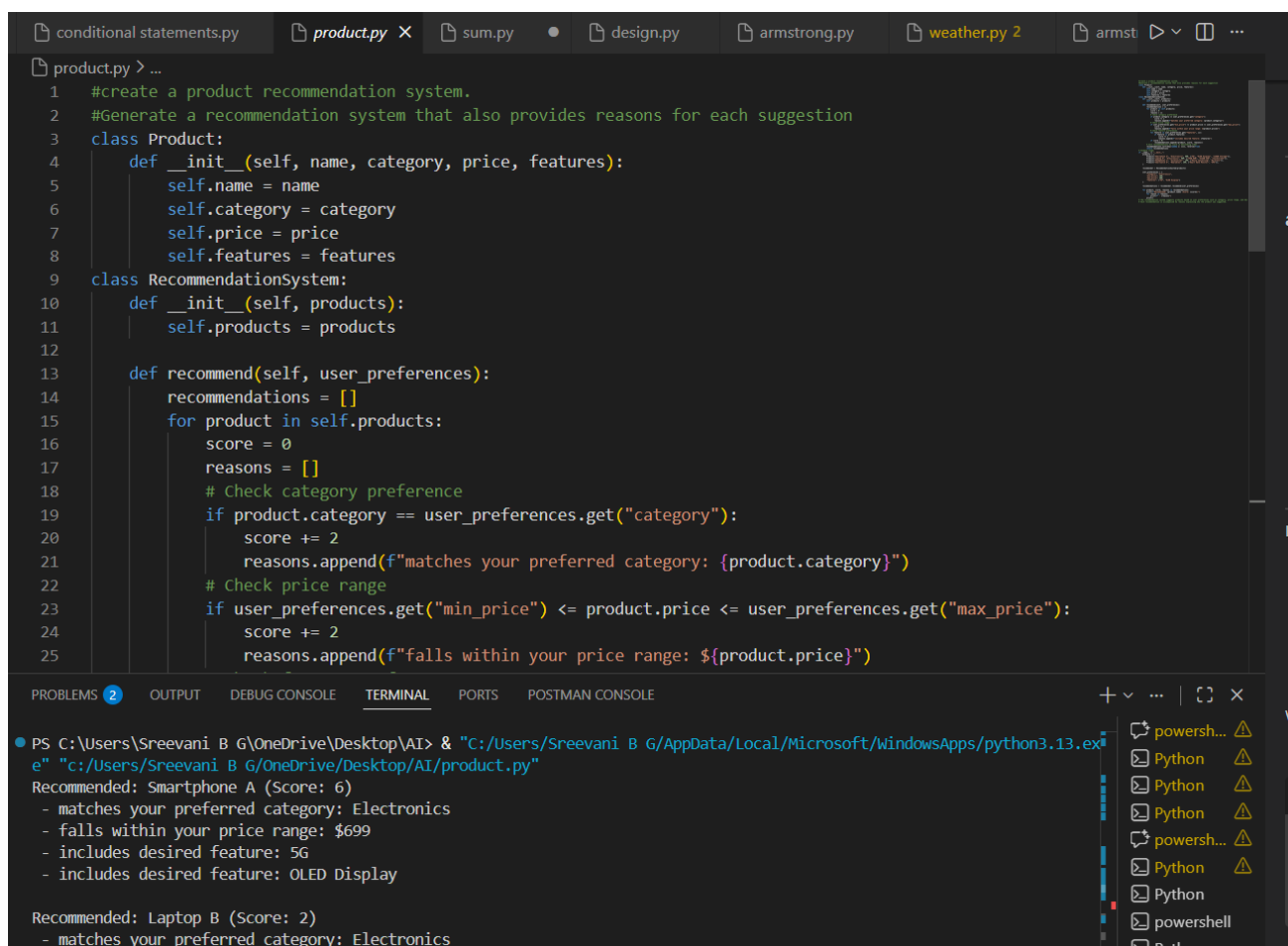
Task: Use AI to create a product recommendation system.

Prompt: "Generate a recommendation system that also provides reasons for each suggestion."

Expected Output:

- Code with explainable recommendations.
- Evaluation of whether explanations are understandable.

Output Screenshot:



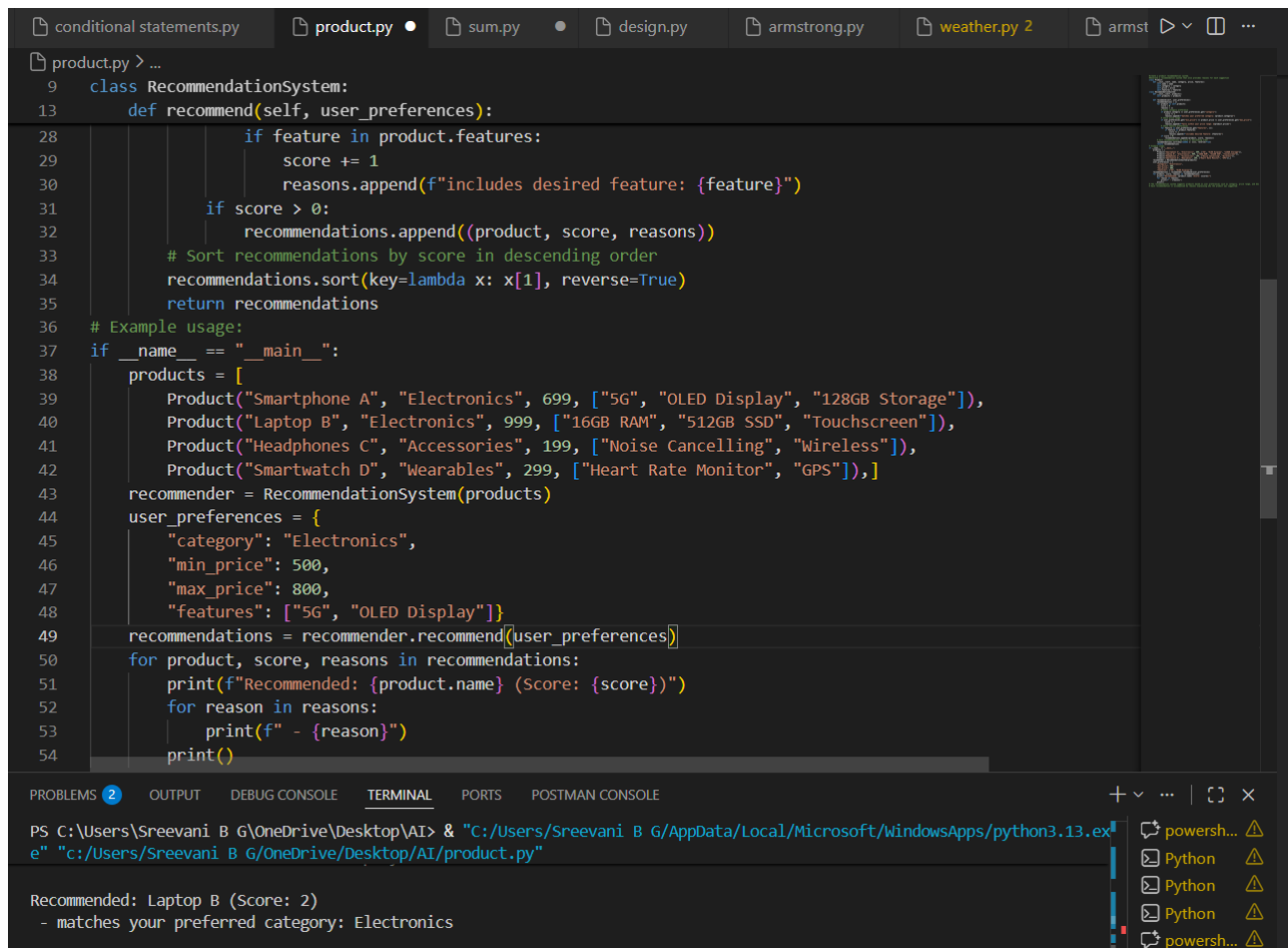
The screenshot displays a Visual Studio Code editor window with a Python file named `product.py` open. The code defines two classes: `Product` and `RecommendationSystem`. The `Product` class has an `__init__` method that initializes `name`, `category`, `price`, and `features`. The `RecommendationSystem` class has an `__init__` method that takes a list of `products` and a `recommend` method that takes `user_preferences`. The `recommend` method iterates through the products, checking if they match the user's preferred category and price range, and appends reasons for each match.

```
1 #create a product recommendation system.
2 #Generate a recommendation system that also provides reasons for each suggestion
3 class Product:
4     def __init__(self, name, category, price, features):
5         self.name = name
6         self.category = category
7         self.price = price
8         self.features = features
9 class RecommendationSystem:
10     def __init__(self, products):
11         self.products = products
12
13     def recommend(self, user_preferences):
14         recommendations = []
15         for product in self.products:
16             score = 0
17             reasons = []
18             # Check category preference
19             if product.category == user_preferences.get("category"):
20                 score += 2
21                 reasons.append(f"matches your preferred category: {product.category}")
22             # Check price range
23             if user_preferences.get("min_price") <= product.price <= user_preferences.get("max_price"):
24                 score += 2
25                 reasons.append(f"falls within your price range: ${product.price}")
```

The terminal output shows the execution of the script, displaying the recommended products and their scores, along with the reasons for each recommendation.

```
PS C:\Users\Sreevani B G\OneDrive\Desktop\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/windowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/Desktop/AI/product.py"
Recommended: Smartphone A (Score: 6)
- matches your preferred category: Electronics
- falls within your price range: $699
- includes desired feature: 5G
- includes desired feature: OLED Display

Recommended: Laptop B (Score: 2)
- matches your preferred category: Electronics
```



```
product.py > ...
9 class RecommendationSystem:
13     def recommend(self, user_preferences):
28         if feature in product.features:
29             score += 1
30             reasons.append(f"includes desired feature: {feature}")
31         if score > 0:
32             recommendations.append((product, score, reasons))
33         # Sort recommendations by score in descending order
34         recommendations.sort(key=lambda x: x[1], reverse=True)
35         return recommendations
36 # Example usage:
37 if __name__ == "__main__":
38     products = [
39         Product("Smartphone A", "Electronics", 699, ["5G", "OLED Display", "128GB Storage"]),
40         Product("Laptop B", "Electronics", 999, ["16GB RAM", "512GB SSD", "Touchscreen"]),
41         Product("Headphones C", "Accessories", 199, ["Noise Cancelling", "Wireless"]),
42         Product("Smartwatch D", "Wearables", 299, ["Heart Rate Monitor", "GPS"]),
43     ]
44     recommender = RecommendationSystem(products)
45     user_preferences = {
46         "category": "Electronics",
47         "min_price": 500,
48         "max_price": 800,
49         "features": ["5G", "OLED Display"]}
50     recommendations = recommender.recommend(user_preferences)
51     for product, score, reasons in recommendations:
52         print(f"Recommended: {product.name} (Score: {score})")
53         for reason in reasons:
54             print(f" - {reason}")
55     print()
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Sreevani B G\OneDrive\Desktop\AI> & "C:/Users/Sreevani B G/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Sreevani B G/OneDrive/Desktop/AI/product.py"

Recommended: Laptop B (Score: 2)
- matches your preferred category: Electronics
```

Evaluation of Explanations

- **Understandability:**

The explanations are clear and human-friendly. They explicitly state:

- The product name.
- Why it matches the user's interest (category alignment).
- Why it fits the budget.
- Why it's considered good (rating + popularity).

- **Transparency:**

Users can see the criteria used (interest, budget, rating, popularity). This avoids the “black box” problem of AI recommendations.

- **Strengths:**

- Easy to read and interpret.
- Shows multiple factors (budget, rating, popularity).
- Provides confidence in why the product was chosen.

- **Possible Improvements:**

- Could add trade-offs (e.g., “slightly higher price but better rating”).
- Could personalize further (e.g., “similar to items you liked before”).